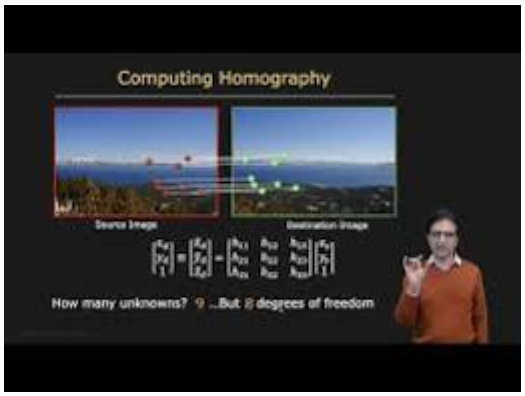
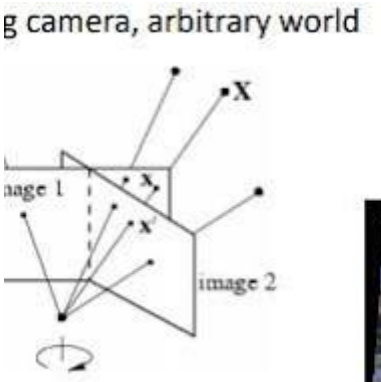
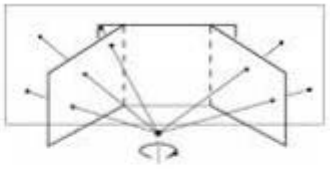






2D transformation hierarchy

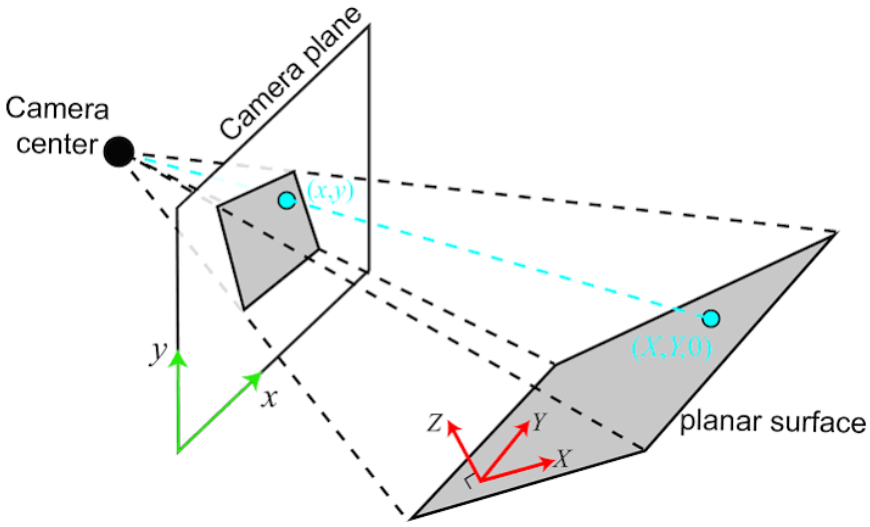
A square transforms to:

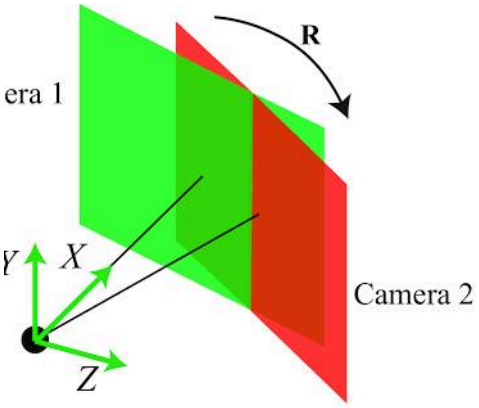
Projective 8dof $\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$

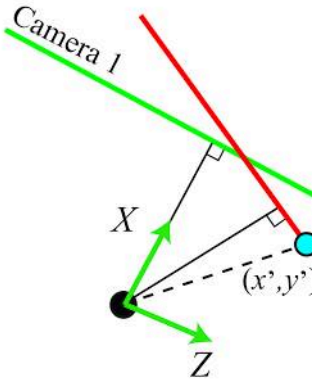
Affine 6dof $\begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$

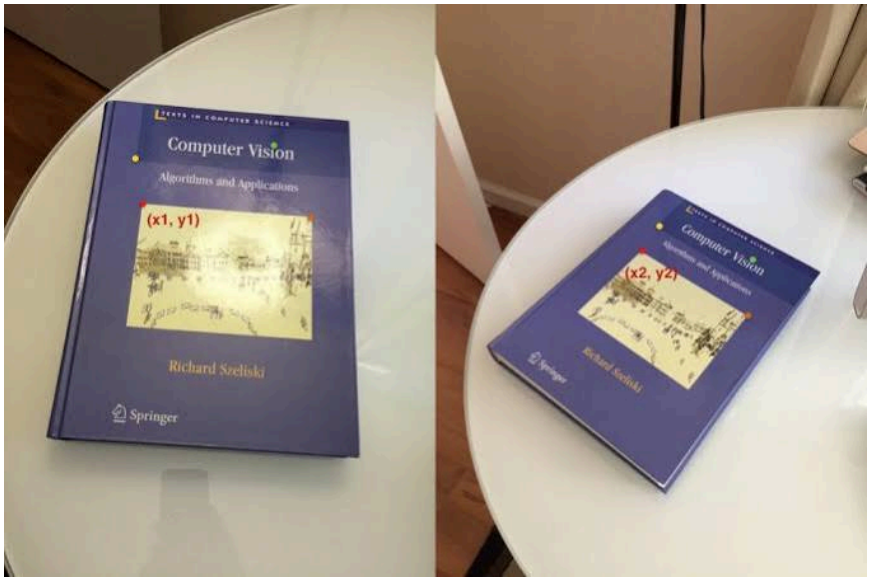
Similarity 4dof $\begin{bmatrix} sr_{11} & sr_{12} & t_x \\ sr_{21} & sr_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$

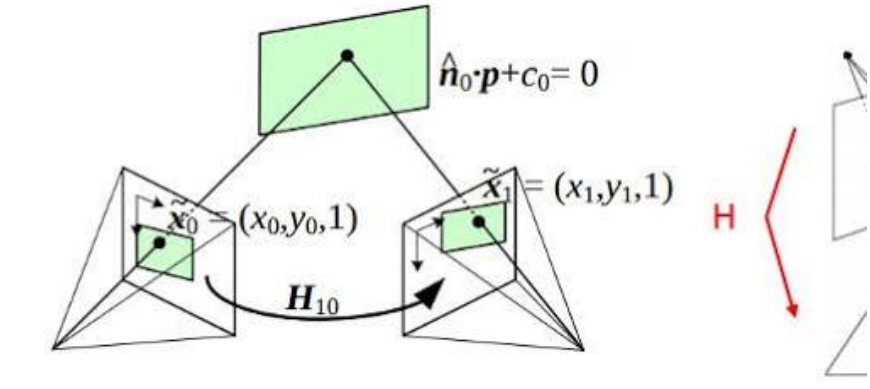
Euclidean 3dof $\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$

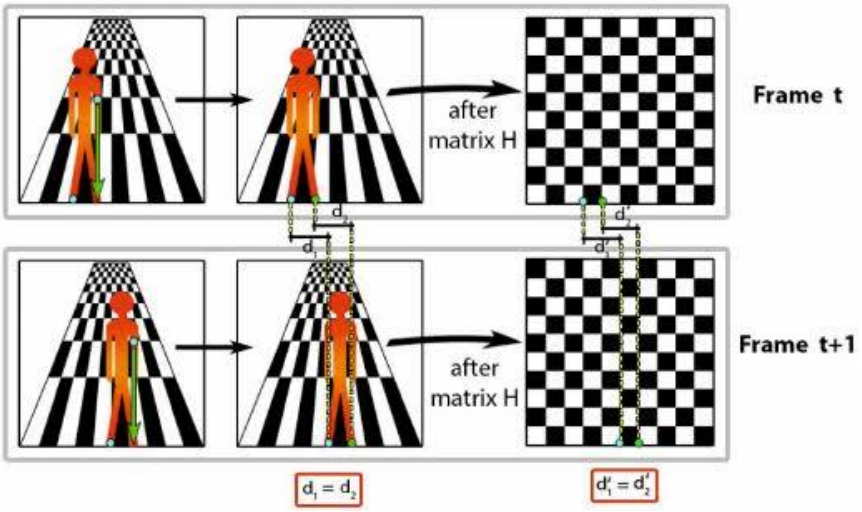

Camera plane
Camera center
(x,y)
y
x
(X,Y,0)
Z
Y
X
planar surface


era 1
R
Y
X
Camera 2
Z


Camera 1
X
(x',y')
Z


Computer Vision
Algorithms and Applications
(x1, y1)
Richard Szeliski
Springer
(x2, y2)


$\hat{n}_0 \cdot p + c_0 = 0$
$\tilde{x}_0 = (x_0, y_0, 1)$
$\tilde{x}_1 = (x_1, y_1, 1)$
H_{10}
H



In computer vision, a **homography** is a geometric transformation matrix that relates two different planar projections of the exact same 3D surface. Represented as a **3x3 matrix (H) with 8 degrees of freedom**, it maps pixels from a source image to a destination image in homogeneous coordinates. Essentially, if you have two pictures of a flat wall taken from different angles—or two pictures taken from a single camera that only rotated without moving—the pixels between those images are perfectly linked by a homography matrix.

Mathematical Representation

Mathematically, a source point (x_1, y_1) maps to a target point (x_2, y_2) using the transformation matrix:

$$\begin{pmatrix} x_2 \\ y_2 \\ 1 \end{pmatrix} = H \cdot \begin{pmatrix} x_1 \\ y_1 \\ 1 \end{pmatrix} = \begin{pmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & 1 \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ y_1 \\ 1 \end{pmatrix}$$

Because the matrix is scale-invariant (defined up to an arbitrary scale factor), the bottom-right value is normalized to 1, leaving **8 independent parameters** to solve.

How Homography is Estimated

To compute a homography matrix, you need a minimum of **4 corresponding point pairs** between the two views:

- **Linear Systems:** Each matching pair provides 2 independent geometric constraints, generating 8 equations to solve the 8 unknown parameters.
- **Algorithms:** It is typically computed using the Direct Linear Transform (DLT) algorithm via Least Squares or Singular Value Decomposition (SVD).
- **Outlier Rejection:** In real-world scenarios, a robust estimator like **RANSAC** (Random Sample Consensus) is applied alongside feature matchers (like SIFT or ORB) to filter out bad pixel matches.
- **Deep Learning Era:** Modern systems also use frameworks like HomographyNet on Emergent Mind to predict displacements directly via supervised or unsupervised neural networks.

Common Applications

- **Image Stitching (Panoramas):** Aligning and warping overlapping photos taken by a rotating camera into a single seamless image.
- **Perspective Correction (Document Scanning):** Straightening an image taken at an angle (like a whiteboard or a receipt) so it looks like it was captured directly from above.
- **Augmented Reality (AR):** Projecting virtual elements (like digital billboards on sports fields or Snapchat filters) onto real-world flat surfaces with perfect perspective.

Are you looking to **implement homography in code** (such as with OpenCV and Python), or do you need help **solving a specific linear algebra equation** for a project?

AI responses may include mistakes. Learn more abou

More